

High-Performance Hardware Implementation of Crystals-Dilithium Based on Improved MDC-NTT

Yijun Cui [✉], *Member, IEEE*, Junjie Zhong [✉], Bei Wang [✉], Tianyu Xu [✉], Chenghua Wang [✉], *Member, IEEE*, and Weiqiang Liu [✉], *Senior Member, IEEE*

Abstract—The growing threat of quantum computing to traditional cryptographic systems has necessitated the development of robust post-quantum algorithms. Crystals-Dilithium, recently standardized by NIST after a three-round competition, is a leading lattice-based digital signature algorithm designed to meet this need. However, conventional hardware implementations of Dilithium often suffer from inefficiencies and performance bottlenecks. To address these weaknesses, this work presents an optimized hardware design for Dilithium across all security levels. The proposed design features a parallel modular multiplication unit, and an enhanced scaling method to reduce bit width and minimize calibration. Additionally, an improved radix-2 Multipath Delay Commutator Number Theoretic Transform (MDC-NTT) and pipelined parallelization using FIFO and BRAM-based buffers are integrated to maximize operating frequency. Evaluated on the Xilinx Artix-7 platform, our implementation achieves a peak frequency of 191 MHz, delivering speedups of 26.3%, 32.5% and 29.6% for key generation, signature generation and signature verification respectively, compared with state-of-the-art works at the highest security level, along with superior hardware efficiency.

Index Terms—Hardware security, post-quantum cryptography, Crystals-Dilithium, FPGA, polynomial multiplier.

I. INTRODUCTION

IN the context of the rapid advancement of science and technology, the vulnerability of information security to various threats has escalated. Presently employed public-key cryptographic schemes predominantly rely on intricate mathematical puzzles characterized by their inherent complexity, rendering them challenging to solve [1]. These cryptographic designs are designed to withstand attacks leveraging classical computer architectures, as the associated mathematical challenges demand

a substantial amount of time for efficient resolution [2], [3]. However, the continuous progression of quantum computer technology poses a formidable threat to these schemes [4]. The non-polynomial time complexity problems they address are susceptible to quantum computer attacks, with Shor's algorithm emerging as a notable risk, capable of solving discrete logarithm and integer factorization problems [5]. In response to these emerging challenges, the National Institute of Standards and Technology (NIST) initiated the post-quantum cryptography standardization process in 2016 [6].

Following the competition of the third round in the selection process, three algorithms—Crystals-Kyber, Crystals-Dilithium and SPHINCS+ have been officially standardized on August 13, 2024 [7]. Crystals-Dilithium, as a digital signature algorithm, is commonly used in scenarios such as verifying identity during digital transactions, encrypted video calls and remotely signing documents. The ongoing assessment of these selected algorithms encompasses various criteria, including their security, key characteristics, ciphertext/signature size and performance in both software and hardware domains [8]. While the development and benchmarking of software implementations are relatively straightforward, the assessment of hardware implementations demands a substantial investment of time and design expertise to gauge their efficiency in terms of speed, area, power consumption and energy efficiency. Particularly, field-programmable gate array (FPGA) has become one of the important platforms for implementing cryptographic algorithms because of its programming flexibility and good performance balance. The hardware implementations of Post-Quantum Cryptography (PQC) on FPGA are deemed essential, providing invaluable insights into the cost and performance considerations associated with these algorithms in hardware contexts.

Extensive investigations into Dilithium digital signature algorithm, encompassing a broad spectrum of security parameters and implementation platforms, have yielded a variety of results. The inaugural implementation of the FPGA for high-performance hardware execution was introduced in [9]. This implementation adopted an optimized parallel hardware structure and judicious allocation of hardware resources, leading to a substantial reduction in cycle count and an augmented overall system operation frequency on the FPGA platform. Beckwith et al. [10] enhanced the memory module of the system through

Received 28 August 2024; revised 20 January 2025; accepted 27 May 2025. Date of publication 4 June 2025; date of current version 11 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62474092, Grant 62134002, and Grant 62104107; and in part by the Natural Science Foundation of Jiangsu Province under Grant BK20210287. Recommended for acceptance by Q. Qiu. (*Corresponding author: Yijun Cui.*)

The authors are with the College of Integrated Circuits, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: yijun.cui@nuaa.edu.cn; zhongjunjie@nuaa.edu.cn; wangbei91@nuaa.edu.cn; xty37603@nuaa.edu.cn; chwang@nuaa.edu.cn; liuweiqiang@nuaa.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3576934

the introduction of a conflict-free memory access method, comprising an address decoder and a 4-bit First-In-First-Out (FIFO) buffer. This method enhanced parallelism by mapping the memory address to a 6-bit binary number and pre-storing it in the memory block in the intended computational order. Consequently, subsequent reads of the memory data occur without encountering any logical conflicts. Land et al. [11] opted to employ a substantial number of DSP units to attain the maximum computational speed for the system. They meticulously refined the Number Theoretic Transform (NTT) structure by judiciously harnessing DSP resources. Zhao et al. [12] proposed a segmented pipelined processing method to reduce both the storage requirements and the processing time, achieving a compact and high-performance hardware architecture. They improved the Multipath Delay Commutator (MDC)-NTT by using four carefully designed butterfly units, and adopted several design trade-offs to achieve a good balance between utilization and performance. Aikata et al. [13] proposed a unified architecture, KaLi, which is the first work that implements a unified cryptoprocessor for Kyber and Dilithium solely in hardware. Gupta et al. [14] designed a lightweight hardware accelerator for Dilithium which utilizes minimal resources in the existing work. They used multiple optimization strategies such as resource and control logic sharing, reuse of modules and pre-calculated LUTs. Li et al. [15] designed a high speed processor for Dilithium based on multiple instruction parallel execution and optimized pseudo random number extension generator. They also proposed a three-stage pipelined modular based on the Karatsuba algorithm, increasing the maximum operating frequency of the system. Mao et al. [16] introduced a sophisticated software-hardware integration approach for Dilithium. Their hardware components are parameterized, enabling comprehensive runtime configuration and enhancing flexibility. Additionally, their innovative software-hardware structure and streamlined operational procedures minimize the data transfer burden between the processor and associated hardware units. Beckwith et al. [17] optimized key modules such as hash module and sampling module based on their previous work. They combined operations with similar data paths but non-overlapping timing, and employed a more compact and flexible Keccak module. Truong et al. [18] presented an efficient low-latency hardware architecture for Dilithium, leveraging optimized timing schedules for its three main algorithms. The hardware implementation enabled runtime switching main operations with various security levels, equipped with designed flexible arithmetic and hash modules.

Moreover, there are many researches specifically for the intensive polynomial multiplication in Dilithium. Du et al. [19] proposed a low-Latency polynomial multiplier. They inserted registers in the Radix-2 MDC structure and designed a five-stage pipelined butterfly unit to increase clock frequency. In addition, through the analysis of the algorithm, they found that the first and second level butterfly unit involve a fixed constant, which is optimized by using addition and shift operations instead of DSP units. Nguyen et al. [20] designed a unique DSP architecture for modular multiplication. Their method reduced the critical path delay of an n -bit multiplier to that of a

$(2n - 2)$ -bit adder, optimizing both area and speed. Zhao et al. [21] employed sparse multiplication instead of NTT polynomial multiplication and achieved a decrease in bit complexity of over 85% and in key storage requirements of over 68%, leveraging the special form of polynomials in Dilithium. On this basis, they proposed a sparse computing unit and a high-speed hybrid architecture, incorporating an efficient scheduling mechanism. Mandal et al. [22] designed a unified and flexible NTT architecture with different numbers of butterfly unit based on Montgomery modular reduction algorithm and proposed an efficient conflict-free memory mapping scheme. Li et al. [23] designed a configurable Karatsuba multiplier that can compute the multiplication of two large numbers for low width factorization and configure multiple butterfly cells in parallel using a loosely coupled architecture. Pham et al. [24] proposed a unique modular multiplication architecture based on Barrett algorithm that can avoid the use of multipliers for large integer multiplication. Chen et al. [25] used an improved Barrett modular multiplication algorithm with hardware-friendly shift and addition operations based on modulus properties and proposed a parallel loop structure that supports arbitrary radix NTT architecture.

However, previous researchs mostly focused on reducing hardware resource consumption to achieve better hardware performance in resource-constrained scenarios or achieve a balance between resource and performance for better hardware efficiency, without providing specific designs tailored for high-speed and high-performance scenarios. To address these weaknesses, we design a high-performance accelerator of Dilithium. The primary contributions of this paper are summarized as follows:

- 1) We propose a novel parallel polynomial multiplication unit combined with a tighter scaling method for high-speed computation. In large integer multiplication, the multiplier is efficiently decomposed into high-order and low-order components, enabling the transformation of the traditionally serial DSP operations into parallel processes. This approach significantly reduces the critical path length, thereby enhancing overall computational speed.
- 2) An improved full pipelined radix-2 MDC-NTT architecture is presented. The activation of each level within the dual-channel commutator unit is governed by an enable signal from the delay register of the preceding stage, ensuring streamlined and efficient data flow. Moreover, the architecture reduces the number of required Block RAMs (BRAMs) and logical address processing units, thereby simplifying the overall hardware structure.
- 3) A unified hardware design for the Dilithium digital signature scheme has been implemented and optimized for the Xilinx Artix-7 FPGA. The proposed design achieves superior performance and hardware efficiency compared to existing solutions. Specifically, our implementation achieves speedups of 26.3%, 32.5%, and 29.6% for key generation, signature generation, and signature verification, respectively, at the highest security level, outperforming the state-of-the-art implementations on the same platform.

TABLE I
PARAMETER SETS

Parameter	level-2	level-3	level-5
N	256	256	256
q -modulus	8380417	8380417	8380417
d -# of dropped bits from $\mathbf{t}$	13	13	13
τ -# of ± 1 's in polynomial $\mathbf{c}$	39	49	60
γ_1 -coefficient range of $\mathbf{y}$	2^{17}	2^{19}	2^{19}
γ_2 -low-order rounding range	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
(k, l) -dimensions of $\mathbf{A}$	(4,4)	(6,5)	(8,7)
η -private key range	2	4	2
$\beta \cdot \tau \cdot \eta$	78	196	120
ω -max # of 1's in the hint $\mathbf{h}$	80	55	75

The subsequent sections of the paper are structured as follows. Section II introduces preliminaries of Crystals-Dilithium Digital Signature and NTT algorithm. Section III introduces the proposed designs, including the high-speed polynomial multiplication unit, the improved pipelined radix-2 MDC-NTT architecture and the top-level hardware system. Section IV analyzes the results and compares with the existing advanced work. Finally, the conclusion is drawn in Section V.

II. PRELIMINARIES

A. Crystals-Dilithium

Crystals-Dilithium is a lattice-based digital signature scheme, based on the Module Learning with Errors (M-LWE) and the Module Short Integer Solution (M-SIS) [26]. The algorithm process includes three stages that are key generation, signature generation and signature verification with three parameter sets detailed in Table I in Version 3.1 of the Dilithium submission [27].

The newest version of Dilithium submission is Module Lattice Digital Signature Algorithm (ML-DSA) in the draft FIPS.204 of NIST [28]. ML-DSA is derived from Version 3.1 of Dilithium, which increases the length of tr to 512 bits and the length of $\tilde{c}$ to 384 and 512 bits, respectively, for the parameter sets level-3 and level-5 [29]. Moreover, there are two variants of the signing algorithm: “hedged” and “deterministic” in ML-DSA while the default version is deterministic in Version 3.1. By default, the “hedged” variant employs fresh randomness. Additionally, an optional deterministic variant is specified for platforms lacking a random number generator. In this paper, the kind of signature we implement is deterministic.

The algorithm of key generation, described in Algorithm 1 does not require any input and is designed to output a public key and a private key. The algorithm initiates by employing an approved random-bit generator (RBG) to generate a 256-bit random seed ξ . This seed is expanded, as required, using SHAKE-256 to yield additional random values. The vector $\mathbf{t}$, in conjunction with the matrix $\mathbf{A}$, constitutes an expanded representation of the public key. The vector $\mathbf{t}$ is compressed in the actual public key by discarding the d least significant bits from each coefficient, resulting in the polynomial vector $\mathbf{t}_1$.

The algorithm of signature generation, described in Algorithm 2 accepts a private key sk and a message M as input while

Algorithm 1: Dilithium.KeyGen.

```

1:  $\xi \leftarrow [0, 1]^{256}$ 
2:  $(\rho, \rho', K) \in [0, 1]^{256} \times [0, 1]^{512} \times [0, 1]^{256} \leftarrow H(\xi, 1024)$ 
3:  $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho)$ 
4:  $(s_1, s_2) \leftarrow \text{ExpandS}(\rho')$ 
5:  $\mathbf{t} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(s_1)) + s_2$ 
6:  $(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}(\mathbf{t}, d)$ 
7:  $tr \leftarrow H(\rho || \mathbf{t}_1, 256)$ 
8: return  $(pk = (\rho, \mathbf{t}_1), sk = (\rho, K, tr, s_1, s_2, \mathbf{t}_0))$ 

```

Algorithm 2: Dilithium.Sign(sk, M).

```

1:  $(\rho, K, tr, s_1, s_2, \mathbf{t}_0) \leftarrow sk$ 
2:  $\hat{s}_1 \leftarrow \text{NTT}(s_1)$ 
3:  $\hat{s}_2 \leftarrow \text{NTT}(s_2)$ 
4:  $\hat{\mathbf{t}}_0 \leftarrow \text{NTT}(\mathbf{t}_0)$ 
5:  $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho)$ 
6:  $\mu \leftarrow H(tr || M, 512)$ 
7:  $\rho' \leftarrow H(K || \mu, 512)$ 
8:  $k \leftarrow 0, (z, \mathbf{h}) \leftarrow \perp$ 
9: while  $(z, \mathbf{h}) = \perp$  do
10:    $\mathbf{y} \leftarrow \text{ExpandMask}(\rho', k)$ 
11:    $\mathbf{w} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{y}))$ 
12:    $\mathbf{w}_1 \leftarrow \text{HighBits}(\mathbf{w})$ 
13:    $\tilde{c} \leftarrow H(\mu || \mathbf{w}_1, 256)$ 
14:    $c \leftarrow \text{SampleInBall}(\tilde{c})$ 
15:    $\hat{c} \leftarrow \text{NTT}(c)$ 
16:    $\langle \langle cs_1 \rangle \rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{s}_1)$ 
17:    $\langle \langle cs_2 \rangle \rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{s}_2)$ 
18:    $\mathbf{z} \leftarrow \mathbf{y} + \langle \langle cs_1 \rangle \rangle$ 
19:    $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{w} - \langle \langle cs_2 \rangle \rangle)$ 
20:   if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$  or  $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$  then  $(z, \mathbf{h}) \leftarrow \perp$ 
21:   else
22:      $\langle \langle ct_0 \rangle \rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{t}}_0)$ 
23:      $\mathbf{h} \leftarrow \text{MakeHint}(-\langle \langle ct_0 \rangle \rangle, \mathbf{w} - \langle \langle cs_2 \rangle \rangle + \langle \langle ct_0 \rangle \rangle)$ 
24:     if  $\|\langle \langle ct_0 \rangle \rangle\|_\infty \geq \gamma_2$  or the number of 1's in  $\mathbf{h}$  is greater than  $\omega$ , then  $(z, \mathbf{h}) \leftarrow \perp$ 
25:   end if
26: end if
27:    $k \leftarrow k + l$ 
28: end while
29: return  $\sigma = (\tilde{c}, z, \mathbf{h})$ 

```

the output is a byte string of the signature σ . Before the message M is signed, it is concatenated with the public-key hash tr and then hashed using the hash function H to generate a 512-bit message representative μ . During each signing operation, the signer generates an additional 512-bit seed ρ' for private randomness. The core of the signing algorithm comprises a rejection sampling loop, iteratively generating either a valid signature or an invalid one whose release would leak information about the private key. The loop is repeated until a valid signature is generated, which is subsequently encoded into a byte string for output. The signer initially generates a “commitment” $\mathbf{w}_1$. Subsequently, the signer pseudorandomly derives a “challenge” c

Algorithm 3: Dilithium.Verify(pk, M, σ).

```

1:  $(\rho, t_1) \leftarrow pk$ 
2:  $(\tilde{c}, z, h) \leftarrow \sigma$ 
3:  $\hat{A} \leftarrow \text{ExpandA}(\rho)$ 
4:  $\mu \leftarrow H(H(\rho || t_1) || M, 512)$ 
5:  $c \leftarrow \text{SampleInBall}(\tilde{c})$ 
6:  $w'_1 \leftarrow \text{UseHint}(h, \text{NTT}^{-1}(\hat{A} \circ \text{NTT}(z) - \text{NTT}(c) \circ \text{NTT}(t_1 \cdot 2^d)), 2\gamma_2)$ 
7:  $\tilde{c}' = H(\mu || w'_1, 256)$ 
8: return  $[[|| z ||_\infty < \gamma_1 - \beta]]$  and  $[[\tilde{c} = \tilde{c}']]$  and  $[[\text{number of 1's in } h \text{ is } \leq \omega]]$ 

```

from w_1 and the message representative μ . Finally, the signer computes a response z .

The algorithm of signature verification, described in Algorithm 3 takes a public key pk , a message M and a signature σ as input while it requires no randomness and outputs a Boolean value indicating the validity of the signature. The verifier reconstructs the commitment w'_1 and proceeds to validate the response z and the hint h from signer. Additionally, they checks that the reconstructed w'_1 is consistent with the commitment hash $\tilde{c}$ from signer. Upon successful completion of all checks, Dilithium.Verify returns true indicating a valid signature. Otherwise it returns false indicating an invalid signature.

B. Number Theoretic Transform

NTT is a variant based on the Fast Fourier Transform (FFT) within an integer modular domain. In the post-quantum cryptography algorithms based on lattices, NTT is commonly employed to expedite polynomial multiplication within the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. The definition of NTT and inverse-NTT (INTT) transformation can be expressed as follows:

$$\text{NTT} : \hat{f}_i = \sum_{j=0}^{n-1} f(j) \omega^{ij} \bmod q \quad (1)$$

$$\text{INTT} : f(i) = \frac{1}{n} \sum_{j=0}^{n-1} \hat{f}_j \omega^{-ij} \bmod q \quad (2)$$

where $i \in [0, n-1]$, ω represents the n -th primitive root of $\mathbb{Z}_q$ satisfying $\omega^n = 1 \bmod q$. Therefore, NTT-based polynomial multiplication can be expressed as the following equation:

$$c = \text{INTT}(\text{NTT}(a) \circ \text{NTT}(b)) \quad (3)$$

where the polynomial a, b, c is on the polynomial ring R_q and $\circ$ denote the point-wise multiplication in the NTT domain. The polynomials a and b are transformed to the NTT domain, then multiplying point by point. Following this, c is reverted to the standard domain by using INTT transformation.

III. PROPOSED HARDWARE IMPLEMENTATIONS

To achieve better performance for the overall Dilithium system, this paper employs a parallel strategy for large integer multiplication, completing the modular multiplication operation

within three clock cycles and uses an improved and tighter scaling method to reduce the bit width of the result that needs to be calibrated. Furthermore, optimizations are applied to the MDC-NTT architecture that utilize preceding delay units to generate enable signals controlling the flipping of subsequent C2 units, significantly reducing the need for combinational logic circuits. Notably, the MDC-NTT structure exhibits high flexibility, enabling multiplication operations across other protocols through splitting the series. Finally, we implement a unified top-level design scheme encompassing three computational stages for all security levels. In this section, we describe the optimized modules and the overall hardware system in detail.

A. The Improved Parallel Modular Multiplication

In the hardware implementation, multiplication of 23×23 bits coefficients is performed using DSP units. However, a DSP unit has a maximum processing capacity of 25×18 bits per operation. In the direct calculation of a 23×23 bits coefficients multiplication, employing two DSP units in series becomes necessary to accommodate the increased bit count. Consequently, the second DSP unit is compelled to wait for the result from the preceding DSP unit before continuing the calculation. This sequential processing decreases the speed of multiplication computation and underutilizes the performance capabilities of the DSP.

In this scenario, we parallelize two DSP units to perform the 23×23 -bit multiplication, enhancing DSP utilization and doubling the speed of coefficient multiplication compared to serial computation [30]. Moreover, we use an improved and tighter scaling method to reduce the bit width of the result that needs to be calibrated. The parallel modular reduction multiplication algorithm employed enables single-cycle multiplication computation through the presplitting of multiplicands. This approach replaces the serial configuration of two DSP units for 23-bit multiplication with a parallel configuration.

Assume that x and y are two elements in $\mathbb{Z}_q$ represented as 23-bit unsigned integers. To multiply them, we initially split the multiplier y into its high 11 bits and low 12 bits, as follows:

$$\begin{aligned}
& x[22:0] \times y[22:0] \\
& \equiv x[22:0] \times (2^{12}y[22:12] + y[11:0]) \\
& \equiv 2^{12}x[22:0] \times y[22:12] + x[22:0] \times y[11:0] \\
& \equiv 2^{12}a[33:0] + b[34:0]
\end{aligned} \quad (4)$$

where a and b can be computed using two DSPs in parallel. Subsequently, the results derived from a and b can be decomposed further by recursively applying the congruence relation $2^{23} \equiv 2^{13} - 1 \bmod 8380417$ as detailed below:

$$\begin{aligned}
& 2^{12}a[33:0] \\
& \equiv 2^{12}(2^{11}a[33:11] + a[10:0]) \\
& \equiv 2^{23}a[33:11] + 2^{12}a[10:0] \\
& \equiv 2^{13}a[33:11] - a[33:11] + 2^{12}a[10:0] \\
& \equiv 2^{13}(2^{10}a[33:21] + a[20:11]) - a[33:11] + 2^{12}a[10:0] \\
& \equiv \dots
\end{aligned}$$

$$\begin{aligned}
&\equiv 2^{13}(a[33:31] + a[30:21] + a[20:11]) \\
&\quad - (a[33:31] + a[33:21] + a[33:11]) + 2^{12}a[10:0] \\
&\equiv r1[25:0] \bmod q \\
&< 2^{13}(a[33:31] + a[30:21] + a[20:11]) + 2^{12}a[10:0] \\
&< 2^{13}(2^3 + 2^{10} + 2^{10}) + 2^{12} \cdot 2^{11} \\
&= 2^{13}(2^3 + 2^{11}) + 2^{23} \\
&= 2^{16} + 2^{24} + 2^{23} \\
&= (2^{16} + 2^{23}) + 2^{24} \\
&< 2^{24} + 2^{24} = 2^{25}
\end{aligned} \tag{5}$$

$$\begin{aligned}
b[34:0] &\equiv 2^{23}b[34:23] + b[22:0] \\
&\equiv 2^{13}b[34:23] - b[34:23] + b[22:0] \\
&\equiv 2^{13}(2^{10}b[34:33] + b[32:23]) - b[34:23] + b[22:0] \\
&\equiv 2^{13}b[34:33] - b[34:33] + 2^{13}b[32:23] \\
&\quad - b[34:23] + b[22:0] \\
&\equiv 2^{13}(b[34:33] + b[32:23]) \\
&\quad - (b[34:33] + b[34:23]) + b[22:0] \\
&\equiv r2[25:0] \bmod q \\
&< 2^{13}(2^2 + 2^{10}) + 2^{23} \\
&= 2^{15} + 2^{23} + 2^{23} \\
&< 2^{25}
\end{aligned} \tag{6}$$

where $r1$ and $r2$ are 26-bit signed integers. For a signed integer, we can also divide it into high bits and low bits by using $2^{23} \equiv 2^{13} - 1 \bmod 8380417$, where the former is a signed integer and the latter is an unsigned integer. Furthermore, the multiplication process can be reduced as follows:

$$\begin{aligned}
&x[22] \times y[22:0] \\
&\equiv r1[25:0] + r2[25:0] \equiv r[26:0] \\
&\equiv 2^{23}r[26:23] + r[22:0] \\
&\equiv 2^{13}r[26:23] - r[26:23] + r[22:0] \\
&\equiv res[24:0] \bmod q
\end{aligned} \tag{7}$$

where r and res are also signed integers. Compared with [30], we use an improved and tighter scaling method to get the result res . $2^{13}r[26:23] - r[26:23]$ is a signed integer whose range is $[-(2^{13} - 1) \times 7, (2^{13} - 1) \times 7]$. $r[22:0]$ is an unsigned integer whose range is $[0, 2^{23} - 1]$. Therefore, res is the sum of the two values whose range is $[-(2^{13} - 1) \times 7, (2^{13} - 1) \times 7 + 2^{23} - 1]$, which is within $(-q, 2q)$. Consequently, an additional peripheral circuit is necessary to confine and calibrate the final result within the desired range. The final result can be obtained easily by at most one addition or subtraction after the reduction process.

The hardware design of the improved modular multiplier unit is depicted in Fig. 1. The left and right DSPs symbolize two data streams that are computed concurrently. After two clock cycles, the results r_1 and r_2 obtained undergo a final operation in the

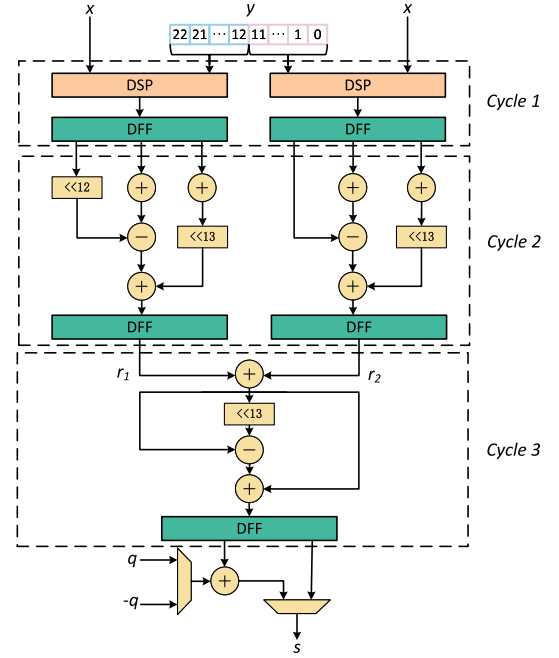


Fig. 1. The schematic of the designed parallel modular multiplication circuit.

third cycle. The results are then utilized to derive accurate multiplication results in the modular domain through the quasi-module comprised of combinational logic.

The improved modular multiplication unit presented in this paper necessitates three clock cycles to complete a computation. The detailed calculation process unfolds as follows. In Cycle 1, the multiplier y is initially divided into the high 11 bits and low 12 bits through a truncation operation. Subsequently, the two sets of data, along with x , are input into the DSP unit for parallel multiplication. Following this, a pipelined segmentation operation is performed using registers. In Cycle 2, a specific modular approximation algorithm $2^{23} \equiv 2^{13} - 1 \bmod 8380417$ is applied to obtain the initial approximation of the two parts of the computation results that are r_1 and r_2 . In Cycle 3, r_1 and r_2 are combined to obtain uncalibrated initial approximation results. Ultimately, additional calibration units are employed to ensure that the final approximation results res conform to the modular domain $[0, q-1]$.

B. The Improved High-Speed Polynomial Multiplier

The radix-2 MDC-NTT architecture is widely employed in pipelined accelerated hardware implementations for lattice-based post-quantum cryptography [31]. In order to enhance the maximum operating frequency and performance, we employ and improve the MDC-NTT architecture. For each level of the initial iteration in the NTT algorithm, we design a unique data processing unit that incorporates eight-level butterfly units. The interconnected data processing unit facilitates continuous and structured data processing, advancing sequentially through $\log_2 n$ levels. The innovative approach aims to substantially improve the efficiency of data processing and operating frequency. The circuit structure, which incorporates two input values, is illustrated in Fig. 2. The *Preprocessing* stage performs

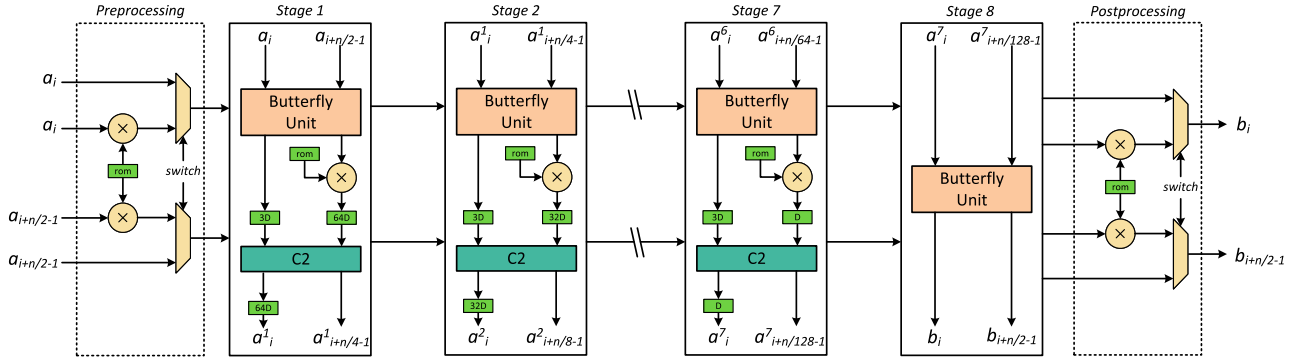


Fig. 2. The schematic of the improved high-speed polynomial multiplier circuit.

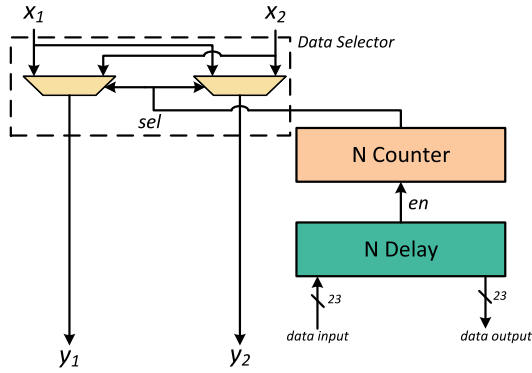


Fig. 3. The schematic of the two-channel commutator unit circuit.

the pre-treatment calculations of the forward NTT, while the *Postprocessing* stage performs the post-treatment calculations of the inverse NTT.

To comply with the specification of MDC-NTT, it is imperative to gradually diminish the distance between two points throughout the data computation process. Consequently, we select the NTT structure using the GS butterfly unit [32]. The comprehensive hardware circuit comprises a forward preprocessing unit, an eight-stage data processing unit and an inverse post-processing unit. Each stage of the data processing unit consists of a radix-2 GS butterfly unit, a data delay unit, a two-channel commutator (C2) unit and a distributed ROM.

The internal configuration of the C2 unit is illustrated in Fig. 3. Two data selectors allocate the dual input data flows to distinct output channels, achieving an efficient data reversal. Each C2 unit incorporates a counter that shares an identical flip-flop cycle, actuated by the preceding N-bit delay element. Detection of input data within the C2 unit prompts the counter to commence counting. Upon reaching the threshold of N , the enabling signal en becomes active, subsequently inverting the data selection signal sel once while maintaining the remainder of the duration unaltered. Streamlining the design, the internal counter is propelled by the output of preceding stage, obviating the necessity for an extraneous logic processing unit for flip-flop signal manipulation. In contrast to external flip-flop logic processors that necessitate recalibration for every variation in flip-flop period to ensure signal accuracy, the value of counter solely necessitates adjustment to accommodate different flip-flop cycles, enhancing overall efficiency and reducing complexity.

In contrast to iterative NTT implementations, pipelined NTT necessitates a greater commitment of resource consumption, but it significantly mitigates the reliance on numerous logical address controllers and BRAMs used to store interim data. Additionally, it easily accommodates NTT computations for diverse points by dynamically adjusting the cascade of modules. Within the pipelined NTT framework, the traditional logical address controllers, that govern the sequence of data computations, are superseded by a combination of data delay units and C2 units, each characterized by different temporal delays. The data delay units meticulously orchestrate timing sequences by strategically introducing delays across varying intervals, leveraging different register counts. Simultaneously, the C2 unit facilitates the rearrangement of output data from the left level, directing the data from initial $n/2$ cycles to the left channel and the data from subsequent $n/2$ cycles to the right channel. Following this, the left channel data undergoes the necessary delay through the corresponding delay unit for the prescribed cycle, while the right channel data is directly dispatched to the butterfly unit of the subsequent stage.

The computation process of MDC-NTT is as follows. The addition result from the butterfly unit in the first data processing stage is directed to the left stream through a three-cycle delay to the two-channel converter, while the subtraction result is directed to the right stream through the multiplier. Subsequently, the output is processed within a register with a capacity of 64. Following a further delay of 64 cycles, this intermediate result merges with another outcome from the C2 unit, triggering data selection and progression to the subsequent stage of data processing. The C2 unit, responsible for data selection for the next level of the data processing unit, functions in conjunction with the convergence of the addition result. In the second data processing stage, the identical delay of 64 cycles is deployed to synchronize the data timing appropriately before calculations, then mirroring the calculation strategy employed in the first stage. The procedural pattern recurs across each subsequent stage, with variations limited to register capacity and the count value of the counter in C2. The register capacity is $n/2^{i+1}$, where $n = 256$ and i represents the number of stage of the butterfly unit. Additionally, a necessary adjustment of the rotation factor is made for each stage, with the rotation factor information stored in the distributed ROM. Furthermore, we incorporate an additional modular multiplication unit externally to execute point-

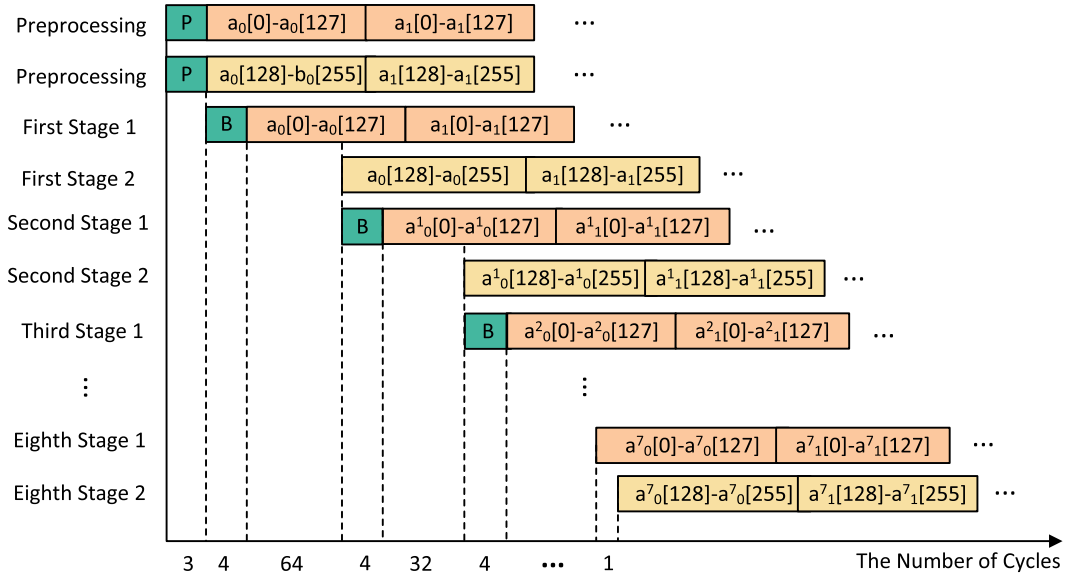


Fig. 4. The time sequence of the improved MDC-NTT architecture.

by-point multiplications, preventing the introduction of additional cycles within the overall system.

The time sequence of improved MDC-NTT is depicted in Fig. 4. The P represents the clock cycle for preprocessing, while B represents the clock cycle for the butterfly unit. The abscissa represents the number of clock cycles and the ordinate represents the different stages of polynomial multiplication. Given that the data path for INTT transform is identical to NTT transform and the function of preprocessing and postprocessing is similar, a complete INTT transform requires an equivalent number of clock cycle as the NTT transform.

The enhanced MDC-NTT structure requires minimal modifications to seamlessly integrate into other post-quantum cryptographic protocols, such as the 12-bit multiplication utilized in Crystals-Kyber. To adapt the computation of modulo multiplication for this purpose, we employ the following steps to translate the 12-bit multiplication in Kyber to a 24-bit multiplication. Initially, we organize the two coefficients of Kyber into distinct groups: the high 12 bits of the data constitute the first coefficient, while the low 12 bits represent the second coefficient. Consequently, these two coefficients collectively form a 24-bit data. The same procedure is applied to derive the second coefficient. Leveraging the law of multiplicative distribution, the multiplication of two 24-bit data results in intermediate data equivalent to the product of four 12-bit data multiplied together. In this case, simplifying the outcome and comparing it with the intermediate result is all that is required to derive the corresponding Kyber result. This approach streamlines the integration of the improved MDC-NTT structure into diverse cryptographic protocols, facilitating its applicability in scenarios such as the 12-bit polynomial multiplication in Crystals-Kyber. Moreover, mask design and out-of-order execution are common protection methods for NTT [33]. These methods can effectively mitigate the threat of side-channel attacks, thus protecting the safety of the circuit.

C. The Hash Module

In Dilithium, the hash module is an important module for generating hash values, which are mainly used to effectively generate hash values of varying lengths for the hashing and protection of public and private keys, and a large number of pseudo-random numbers for polynomial multiplication operation. Hash function is a type of function that transforms an input message string of variable length into a fixed long output string, making it challenging to reverse-engineer the input string from the output string. SHA-3, a cryptographic hash function, is identified by NIST through a standardization competition [34], which is widely applied in post-quantum cryptography. The Dilithium digital signature algorithm employs the extensible functions SHAKE-128 and SHAKE-256 of the SHA-3 which is based on the Keccak permutation [35] for polynomial coefficients and protected hash values.

The architecture of the hash module is illustrated in Fig. 5. The 64-bit input message M is serialized and fed into the hash module, with the length d of the stored data is assigned to the subtraction counter as an initial value before the operation. Once the counter reaches zero, the data is processed and filled, and subsequently input into the shift register. The key null and full condition of the shift register depends on the configured bit rate parameter. The absorption stage is completed by performing logic circuit operations on the iterated r -bit data within the Keccak function. During the extrusion stage, the Keccak function stores the output data in the register and converts parallel data to serial output through the characteristics of the shift register. The signal *done* serves as a flag bit. When the output data length meets the calculation demand, the signal *done* is triggered, and the internal register is cleared synchronously based on this flag. The whole circuit can complete the calculation of SHAKE-128 or SHAKE-256 function through the control signal.

D. Top Designs

The overall architecture of Dilithium system is illustrated in Fig. 6, containing polynomial multiplication modules, sample

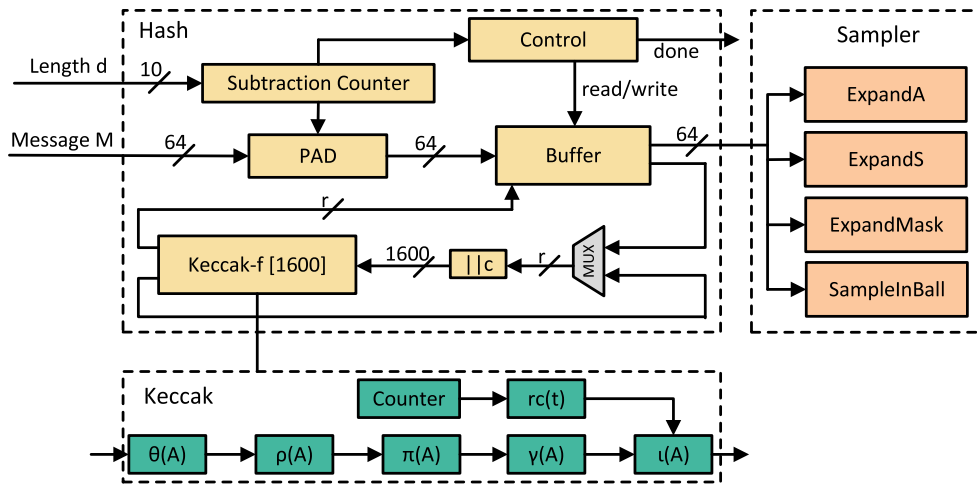


Fig. 5. The schematic of the hash module.

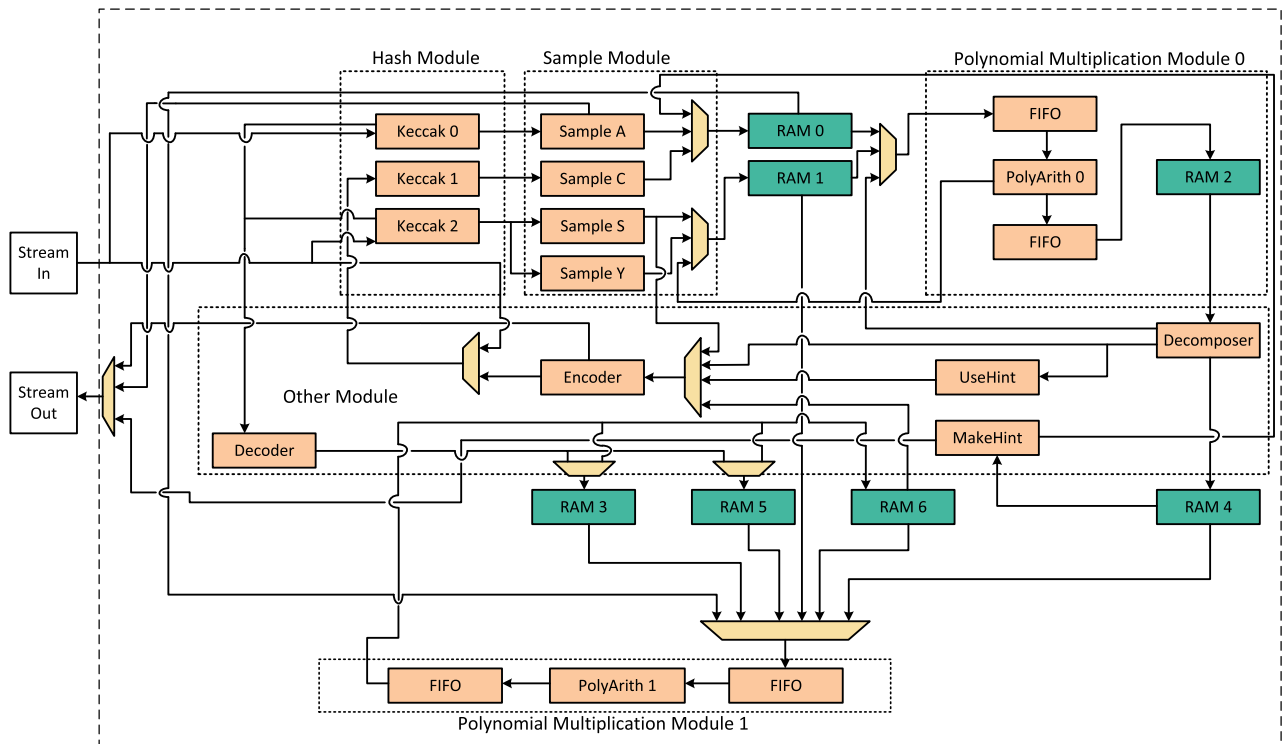


Fig. 6. The schematic of the top-level overall Dilithium system circuit.

modules, hash modules and other functional submodules. The combined architecture is chosen due to the variations in security levels of the Dilithium digital signature, which are primarily determined by differences in vectors and matrix dimensions, as well as the number of coefficients. This necessitates configuring varying sizes of BRAM in the hardware design. By adopting a combined architecture, we streamline the implementation, requiring consideration for only a single BRAM. To enhance the utilization of the BRAM, we employ distinct BRAM address control logic for different security levels. Consequently, we strategically utilize the less occupied portions of BRAM to store a combination of precomputation and postcomputation data, along

with intermediate data. The strategic utilization of resources contributes to the overall efficiency of the system.

The architecture employs a 96-bit data line width, accommodating the storage of four coefficients per line. This configuration effectively enhances data throughput for various computational tasks and matches the frequency of polynomial multiplier. Since the polynomial multiplier processes two coefficients in parallel, the serial-to-parallel converter is essential. This converter, primarily composed of FIFO structures, is integrated both before and after each polynomial multiplier. Meanwhile, to pursue higher performance, we elaborately employ two polynomial multipliers to complete polynomial multiplication.

Dilithium necessitates a significant volume of pseudorandom data for polynomial sampling. Especially, at the security level 5, the matrix A possesses dimensions of 8×7 , where each sample necessitates 24 bits of pseudorandom data, producing amount of pseudorandomness. To prevent sampling from becoming a performance bottleneck in high-performance designs, we employ three Keccak cores primarily dedicated to hash values and matrix sampling. It's worth noting that Keccak samplers, external inputs and outputs are represented by 64 bits in Fig. 5. This consideration ensures compatibility and consistency in data representation within the overall architecture.

It is worth noting that the system is vulnerable to side channel attacks that relies on collecting the physical characteristics of the cryptosystem running time [36]. The common side channel attack techniques include energy attack, time attack, electromagnetic attack and fault attack. Ravi et al. [37], [38], [39] employed side channel attack technique to attack lattice-based post-quantum cryptographic schemes. They proposed an alternative signing procedure, through which it is possible to forge signatures in Dilithium with only the extracted portion of the secret-key without requiring the knowledge of all its elements. Mujdei et al. [40] proposed a generic non-profiled side-channel attack methodology targeting all the different polynomial multiplication algorithms used in lattice-based cryptography. Liu et al. [41], [42] used the randomness leakage attacks to obtain the secret key. Jendral et al. [43] presented a fault injection attack on an implementation of Dilithium in hedged mode that requires only a single faulty signature to recover the secret key vector s_1 . In addition, there are some researches that combine machine learning with side-channel attacks [44]. Han et al. [45] designed a single-trace attack on Dilithium using machine learning-based profiling. They can recover the full secret key from a single power trace taken from the implementation of the NTT in signing procedure. Li et al. [46] presented an enhanced machine learning attack on LWE with sparse binary secrets, which recovers secrets in much larger dimensions and with larger Hamming weights. To make the system we proposed resist to side-channel attacks effectively, we can incorporate specific masking or out-of-order execution technologies in the polynomial multiplication [33]. Since the focus of this paper is still on the high-performance design for Dilithium, it will be considered in the future work.

IV. RESULTS AND ANALYSIS

To comprehensive evaluate the proposed overall system of Dilithium, experiments are implemented on the Xilinx Artix-7 on Vivado 2020.2. Table II reports the resource utilization of submodules used in our design.

Our design adopts the improved full pipelined MDC-NTT for polynomial multiplier and parallel modular reduction, significantly shortening the critical path and reducing the demand for BRAM resources. This adaptation enables performance gains on certain small memory devices. Simultaneously, the pipelined approach imbues the design with modularity, enhancing the scalability and generalizability of the multiplier. In scenarios involving large fan-in or fan-out applications, scalability is achieved by simply increasing the number of pipelines, eliminating the need for designing additional control and address cells. Furthermore,

TABLE II
RESOURCE UTILIZATION OF DILITHIUM SUBMODULES IN THE COMBINED ARCHITECTURE

Submodule	Resource Utilization			
	LUT	FF	DSP	BRAM
96 × 4096 RAM	0	0	0	11
NTT	4187×2	4387×2	18×2	0
MakeHint	2390	730	0	0
UseHint	6453	2800	0	0
Encoder	1570	435	0	0
Decoder	2231	247	0	0
Decomposer	1449	593	0	0
SampleA	1793	667	0	0
SampleY	2094	608	0	0
SampleS	1760	388	0	0
SampleC	1746	832	0	0
Keccak	5483×3	4421×3	0	0
Point-by-Point Mul	591	570	2	0
Other	5988	1306	0	0
Combined	52888	31213	38	11

the pipelined design does not necessitate additional global control signals or internal logic address control. Only a modification of the input data stream's bit length is required for widespread adoption within any compliant overall architecture. The distinctive advantage of our design, in comparison to the conventional single-module iterative type, lies in its ability to attain higher frequencies and performance levels. This achievement comes with minimal design complexity and exceptional application flexibility, marking a notable improvement over existing studies. Moreover, to minimize operation time, we parallelize two NTT polynomial multiplication and three Keccak cores. While this increases resource consumption, our objective remains focused on attaining remarkable performance within permissible resource utilization limits, especially for high-performance scenarios.

The implementation results of NTT architecture and the comparison with existing works on the Xilinx Artix-7 are detailed in Table III. The optimization directions of design methods vary depending on the specific application scenarios. To facilitate a fair and comprehensive comparison of different hardware designs, we introduce a unified evaluation index area and time product (ATP) to measure the hardware efficiency quantitatively. For fair comparison, DSPs and BRAMs are converted to an equivalent number of Slices. The equivalent number of Slices (ENS) = Slices + DSPs × 100 + BRAMs × 196 [47], where Slices are estimated by the formula LUTs/4 as one Slice consists of four LUTs on the Xilinx Artix-7 [48]. Moreover, ATP = ENS × Time (total), which is a representation of hardware efficiency, and the smaller ATP represents the preferable hardware efficiency of the implementation results.

When making a fair comparison, the number of butterfly units should be taken into account. We compare our design with existing works that use eight butterfly units for high performance. Compared with other works on the same platform, our design has the best hardware efficiency while achieving high performance. Compared with [19] and [49], our design achieves speedups of 69.5% and 13.5%, and reduces the ATP in 71.0% and 78.6% respectively. Compared with [15], our speed is comparable to theirs, but operating frequency has increased 10.0% and the ATP is reduced by 39.6%.

TABLE III
IMPLEMENTATION RESULTS OF NTT ARCHITECTURE AND COMPARISON WITH EXISTING WORKS ON THE XILINX ARTIX-7 FPGA

Design	LUT	FF	DSP	BRAM	ENS ¹	BFs	Max.Freq (MHz)	Cycles	Time (μ s)	ATP ² (ENS $\cdot \mu$ s)
[15]	7893	3512	24	2	4765	8	270	256	0.95	4527
[19]	4040	3195	12	4	2994	8	151	476	3.15	9431
[49]	11000	5182	64	12	11502	8	140	156	1.11	12767
this work	4187	4387	18	0	2847	8	297	286	0.96	2733

¹ Equivalent number of slices (ENS) = Slices + DSPs $\times$ 100 + BRAMs $\times$ 196, where Slices = LUTs/4 on the Xilinx Artix-7.

² Area and time product (ATP) = ENS $\times$ Time.

TABLE IV
RESULTS AND COMPARISON WITH EXISTING IMPLEMENTATIONS FOR DILITHIUM SCHEME ON THE XILINX ARTIX-7 FPGA

Design	Level	LUT	FF	DSP	BRAM	ENS ¹	Max.Freq (MHz)	Cycles ² (k)	Time ² (μ s)	Speedup ³	ATP ⁴ (ENS $\cdot$ s)
[10]	2							4.9/29.9/6.6	42.2/257.8/56.9	40.1%/41.8%/52.2%	7.35
	3	53187	28318	16	29	20581	116	8.3/49.4/9.7	71.6/425.9/83.6	42.8%/40.2%/46.8%	11.96
	5							14.0/55.0/13.6	120.7/474.1/117.2	48.8%/39.6%/44.2%	14.65
[11]	2	27433	10681	45	15	14298	163	18.8/76.6/19.7	115.3/469.9/120.9	78.2%/68.1%/77.5%	10.10
	3	30900	11372	45	21	16341	145	33.1/123.2/32.1	228.3/849.7/221.4	81.9%/70.0%/79.9%	21.23
	5	44653	13814	45	31	21739	140	51.0/145.9/52.7	364.3/1042.1/376.4	83.0%/72.5%/82.6%	38.76
[12]	2							4.2/31.6/4.4	43.3/326.1/45.4	42.0%/54.0%/40.1%	4.42
	3	29998	10366	10	11	10656	96.9	5.9/49.5/6.1	60.9/510.8/63.0	32.0%/50.2%/29.4%	6.76
	5							8.8/55.3/9.0	90.1/570.7/92.9	31.4%/49.8%/29.6%	8.03
[14]	5	13975	6845	4	35	10754	163	63.2/113.9/67.9	387/699/416	84.0%/59.0%/84.3%	16.15
[15]	2							4.4/30.1/5.0	33.9/231.5/38.5	26.0%/32.2%/29.4%	4.02
	3	32320	9734	24	14	13224	130	7.3/49.5/8.0	56.1/380.8/61.6	26.2%/33.2%/27.8%	6.59
	5							10.9/55.2/12.7	83.8/424.6/97.6	26.3%/32.5%/33.0%	8.01
[21]	2							3.9/12.8/4.2	32.8/107.6/35.3	23.5%/-39.5%/22.9%	2.58
	3	30304	13559	16	28	14664	119	6.5/21.4/6.9	54.6/179.8/58.0	24.2%/-41.5%/23.3%	4.29
	5							10.9/27.6/11.5	91.6/231.9/96.6	32.5%/-23.5%/32.3%	6.16
this work	2							4.8/28.8/5.2	25.1/150.1/27.2		3.88
	3	52888	31213	38	11	19178	191	7.9/48.6/8.5	41.4/254.5/44.5	-	6.53
	5							11.8/54.7/12.5	61.8/286.4/65.4		7.93

¹ Equivalent number of slices (ENS) = Slices + DSPs $\times$ 100 + BRAMs $\times$ 196, where Slices = LUTs/4 on the Xilinx Artix-7.

² The values refer to key generation, signature generation and signature verification respectively.

³ The values refer to the speedup of this work compared with the references.

⁴ Area and time product (ATP) = ENS $\times$ Time (total).

The results and comparison with existing implementations on the Xilinx Artix-7 are detailed in Table IV. The values of Cycles and Time refer to the cycles and time consumed for key generation, signature generation, and signature verification, respectively. The values of Speedup refer to the speedup of this work compared with the references for key generation, signature generation and signature verification respectively. The superiority of our design in terms of frequency performance is evident. Compared with advanced designs, our implementation achieves the highest maximum operating frequency and minimum execution time, presenting the best performance.

Compared with the designs in [10], we employ an improved pipelined MDC-NTT architecture instead of the iterative NTT architecture. Iterative designs entail a substantial amount of logic circuits, limiting the maximum frequency compared with fully pipelined approaches. In the top-level system, we similarly employ two polynomial multiplication modules and three hash modules to shorten the clock cycles. Our overall design achieves an operating frequency of 191 MHz, while theirs is only 116 MHz, presenting a remarkable improvement in performance. Moreover, we achieve significant speedups of 48.8%, 39.6%

and 44.2% for key generation, signature generation and signature verification respectively at the security level 5.

To achieve high computational speed, Land et al. [11] employ a significant number of DSP units for the polynomial multiplication. Compared with their overall design, ours utilizes fewer DSPs and optimizes the BRAM utilization to minimize overall resource consumption. However, since only one Keccak core is used in their design, the sampling of matrix **A** consumes 33,767 clock cycles at the security level 5, comprising approximately two-thirds of the key generation and the remaining sampling processes must await completion of sampling of matrix **A** before continuing. Consequently, the overall performance is hindered by the significant number of clock cycles consumed by the sampling module. Our optimized designs of hash modules avoid similar problems and significantly reduce the clock cycles required in the overall system. Compared with [11], we only have a minimal increase in consumption of LUTs and FFs, while achieving significant speedups of 83.0%, 72.5% and 82.6% for key generation, signature generation and signature verification respectively at the security level 5.

The works of [12] and [14] present compact and lightweight designs for Dilithium. The implementation in [12] achieves a good balance between resource utilization and performance, using various design trade-offs. Compared with their designed MDC-NTT, our improved fully pipelined MDC-NTT exhibits a higher operating frequency using parallel modular reduction that matches the holistic frequency. Our design uses the preceding delay unit to produce an enable signal, which governs the toggling of subsequent C2 units, thereby significantly reducing the need for combinational logic circuitry. Our design is $1.97\times$ of [12] in frequency and reduces execution time effectively. Gupta et al. [14] design a lightweight hardware accelerator for Dilithium only at the security level 5, which utilizes minimal resources in the existing work. Although they reach a respectable frequency at 163 MHz, their design consumes a large number of clock cycles, resulting in poor performance. Their designs are suitable for resource-constrained scenarios, while ours are more suitable for high-performance scenarios.

The design of [15] presents the best performance in the existing work. They propose a full pipelined NTT architecture based on the Karatsuba algorithm, utilizing 7893 LUTs, 3512 FFs, 24 DSPs and 2 BRAMs while our design achieves 47.0% and 25.0% decrease in LUT and DSP without using BRAMs. However, employing a single NTT module increases the cycle of polynomial multiplication, restricting the performance of the overall system. Compared with [15], at the highest security level, our implementation achieves speedups of 26.3%, 32.5% and 33.0% for key generation, signature generation and signature verification respectively.

According to ATPs, our designed architecture has a similar resource consumption with [10] and [11] at the highest security level, but reduces the ATP in 45.87% and 79.54% respectively. Compared with works of [12] and [14] that designed for resource-constrained scenarios, our design has a fewer ATP with [12] and reduces the ATP in 50.90% with [14], presenting better hardware efficiency. Compared with [15], although using more resource consumption, we achieve better performance while having better hardware efficiency. Therefore, the performance and hardware efficiency of designed Dilithium architecture are significantly improved compared to the state-of-the-art works at all three different security levels.

V. CONCLUSION AND FUTURE WORK

In this work, we introduce a high-performance hardware implementation of the Crystals-Dilithium digital signature scheme. Leveraging an improved parallel modular reduction algorithm and a pipelined MDC-NTT polynomial multiplier structure, our design achieves exceptional performance on the Xilinx Artix-7. The versatility of the design is demonstrated by its ability to meet the requirements of all three security levels of Dilithium, with the three computation processes seamlessly integrated. Compared with existing works for Dilithium on the same platform, the performance of our design is the best, achieving the maximum operating frequency and the minimum execution time. Our implementation achieves speedups of 26.0%-84.0%, 32.2%-72.5% and 27.8%-84.3% for key generation, signature generation and signature verification respectively

at the different security levels. Moreover, it can be seen the architecture designed in this work is the best for hardware efficiency among existing works, achieving the smallest ATP.

In the subsequent research, we need to optimize the designs of various modules and the timing of the overall system to reduce operation cycles. From the performance analysis, it can be seen that the high speed of our design benefits from the high operating frequency. However, compared with other works, the overall operation cycles has no advantage which is a shortcoming required to be improved. Additionally, the polynomial multiplier with good expansibility designed in this paper can be combined with Kyber algorithm for joint implementation. Furthermore, NIST conducted PQC Additional Digital Signature Schemes Round 1 in 2023 where Raccoon is a promising scheme [50], [51]. The security of Raccoon relies on standard assumptions on module lattices, which are the same one used in the selected standard Dilithium. In the Raccoon, there are the parameter $n = 512$ and the modulus $q = 549824583172097$ which is 49-bit. The $2n$ -th root of unity $g = 358453792785495$ satisfying $g^{2n} = 1 \bmod q$. For polynomial multiplication accelerated using the NTT algorithm, an architecture similar to the one designed in this paper can be employed. The NTT architecture can be extended to $\log_2 n = 9$ levels, and a similar approach can be employed for modular multiplication, leveraging the congruence relation of specific modules. It is worth noting that we need to consider the security threats posed by side-channel attacks. Although side-channel analysis and countermeasures are not been considered and the system is not been protected through TVLAs, we will carry out these works such as incorporating specific masking or out-of-order execution technologies in our design and completing side-channel analysis in the future.

REFERENCES

- [1] C. Peikert, "Public-key cryptosystems from the worst-case shortest vector problem: Extended abstract," in *Proc. 41st Annu. ACM Symp. Theory Comput. (STOC)*, New York, NY, USA: ACM, 2009, pp. 333–342.
- [2] L. Chen et al., "Report on post-quantum cryptography," Gaithersburg, MD, USA: US Department of Commerce, National Institute of Standards and Technology, vol. 12, 2016.
- [3] D. Micciancio and O. Regev, *Lattice-Based Cryptography*. Berlin, Germany: Springer Berlin Heidelberg, 2009, pp. 147–191.
- [4] E. E. Targhi and D. Unruh, "Post-quantum security of the Fujisaki-Okamoto and OAEP transforms," in *Theory of Cryptography*, M. Hirt and A. Smith, Eds., Berlin, Germany: Springer Berlin Heidelberg, 2016, pp. 192–216.
- [5] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, pp. 1484–1509, Oct. 1997.
- [6] G. Alagic et al., "Status report on the third round of the NIST post-quantum cryptography standardization process," 2022, pp. 00–202.
- [7] Saarinen, M. Juhani, and D. Smith-Tone, "Post-quantum cryptography," in *15th Int. Workshop, PQCrypto*, 2024.
- [8] D. Soni and R. Karri, "Efficient hardware implementation of PQC primitives and PQC algorithms using high-level synthesis," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2021, pp. 296–301.
- [9] S. Ricci et al., "Implementing CRYSTALS-Dilithium signature scheme on FPGAs," in *Proc. 16th Int. Conf. Availability, Rel. Secur. (ARES)*, New York, NY, USA: ACM, 2021, pp. 1–11.
- [10] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of CRYSTALS-Dilithium," in *Proc. Int. Conf. Field-Programmable Technol. (ICFPT)*, 2021, pp. 1–10.
- [11] G. Land, P. Sasdrich, and T. Güneysu, "A hard crystal - Implementing Dilithium on reconfigurable hardware," in *Proc. 20th Int. Conf. (CARDIS) Smart Card Res. Adv. Appl.*, Lübeck, Germany, Berlin, Germany: Springer-Verlag, 2021, pp. 210–230.

- [12] C. Zhao et al., "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2022, no. 1, pp. 270–295, 2022.
- [13] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, "KaLi: A crystal for post-quantum security using Kyber and Dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 747–758, Feb. 2023.
- [14] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-Dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 8, pp. 3234–3243, Aug. 2023.
- [15] X. Li, J. Lu, D. Liu, A. Li, S. Yang, and T. Huang, "A high speed post-quantum crypto-processor for crystals-Dilithium," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2024.
- [16] G. Mao et al., "High-performance and configurable SW/HW co-design of post-quantum signature CRYSTALS-Dilithium," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 3, pp. 1–28, 2023.
- [17] L. Beckwith, D. T. Nguyen, and K. Gaj, "Hardware Accelerators for Digital Signature Algorithms Dilithium and FALCON," *IEEE Des. Test; IEEE Des. Test Comput.*, vol. 41, no. 5, pp. 27–35, 2024.
- [18] Q. D. Truong, P. N. Duong, and H. Lee, "Efficient low-latency hardware architecture for module-lattice-based digital signature standard," *IEEE Access*, vol. 12, pp. 32395–32407, 2024.
- [19] G. Du, Z. Chen, Z. Li, X. Wang, and D. Zhang, "A Low-latency polynomial multiplier accelerator for CRYSTALS-Dilithium digital signature," in *Proc. Great Lakes Symp. VLSI (GLSVLSI)*, New York, NY, USA: ACM, 2024, pp. 258–262.
- [20] T.-H. Nguyen, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "High-speed NTT accelerator for CRYSTAL-Kyber and CRYSTAL-Dilithium," *IEEE Access*, vol. 12, pp. 34918–34930, 2024.
- [21] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for CRYSTALS-Dilithium," in *Proc. IEEE Int. Symp. Hardware Oriented Secur. Trust (HOST)*, 2024, pp. 150–159.
- [22] S. Mandal and D. B. Roy, "KiD: A hardware design framework targeting unified NTT multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA," in *Proc. 37th Int. Conf. VLSI Des. 23rd Int. Conf. Embedded Syst. (VLSID)*, 2024, pp. 455–460.
- [23] B. Li, Y. Yan, Y. Wei, and H. Han, "Scalable and parallel optimization of the number theoretic transform based on FPGA," *IEEE Trans. Very Large Scale Integr.*, vol. 32, no. 2, pp. 291–304, Feb. 2024.
- [24] T. X. Pham, P. Duong-Ngoc, and H. Lee, "An efficient unified polynomial arithmetic unit for CRYSTALS-Dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4854–4864, Dec. 2023.
- [25] X. Chen, B. Yang, Y. Lu, S. Yin, S. Wei, and L. Liu, "Efficient access scheme for multi-bank based NTT architecture through conflict graph," in *Proc. 59th ACM/IEEE Des. Automat. Conf. (DAC)*, New York, NY, USA: ACM, 2022, pp. 91–96.
- [26] L. Ducas et al., "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *Trans. Cryptographic Hardware Embedded Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.
- [27] S. Bai et al., "CRYSTALS-Dilithium algorithm specifications and supporting documentation (Version 3.1)," NIST Post-Quantum Cryptography Standardization Round 3, 2021.
- [28] "Raimondo, M. Gina, and E. L. Laurie, module-lattice-based digital signature standard," FIPS 204 Standard, 2023.
- [29] C. Cremers, S. Dölzl, R. Fiedler, M. Fischlin, and C. Janson, "Buffing signature schemes beyond unforgeability and the case of post-quantum signatures," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 1696–1714.
- [30] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient Implementation of Dilithium signature scheme on FPGA SoC platform," *IEEE Trans. Very Large Scale Integr.*, vol. 30, no. 9, pp. 1158–1171, Sep. 2022.
- [31] C. P. Renteria-Mejía and J. Velasco-Medina, "High-throughput ring-LWE cryptoprocessors," *IEEE Trans. Very Large Scale Integr.*, vol. 25, no. 8, pp. 2332–2345, Aug. 2017.
- [32] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. Fall Joint Comput. Conf. (AFIPS)*, New York, NY, USA: ACM, 1966, pp. 563–578.
- [33] Z. Chen, Y. Ma, and J. Jing, "Low-cost shuffling countermeasures against side-channel attacks for NTT-based post-quantum cryptography," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 1, pp. 322–326, Jan. 2023.
- [34] M. J. Dworkin, "SHA-3 standard: Permutation-based hash and extendable-output functions," NIST, 2015.
- [35] G. Bertoni, J. Daemen, and M. Peeters, "Keccak sponge function family main document," Submission to NIST (Round 2), vol. 3, pp. 320–327, 2009.
- [36] H. Steffen, G. Land, L. Kogelheide, T. Güneysu, T. Johansson, and D. Smith-Tone, "Breaking and protecting the crystal: Side-channel analysis of Dilithium in hardware," in *Post-Quantum Cryptography*, Cham, Switzerland: Springer Nature Switzerland, 2023, pp. 688–711.
- [37] P. Ravi, M. P. Jhanwar, J. Howe, A. Chattopadhyay, and S. Bhasin, "Side-channel assisted existential forgery attack on Dilithium - A NIST PQC candidate," Cryptology ePrint Archive, Paper 2018/821, 2018.
- [38] P. Ravi, S. Bhasin, S. S. Roy, and A. Chattopadhyay, "On exploiting message leakage in (few) NIST PQC candidates for practical message recovery attacks," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 684–699, 2022.
- [39] P. V. Ravi, M. F. Ezerman, S. Bhasin, A. Chattopadhyay, and S. S. Roy, "Will you cross the threshold for me? Generic side-channel assisted chosen-ciphertext attacks on NTRU-based KEMs," *IACR Trans. Cryptogr. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 722–761, 2022.
- [40] C. Mudei, L. Wouters, A. Karmakar, A. Beckers, J. M. Bermudo Mera, and I. Verbauwhede, "Side-channel analysis of lattice-based post-quantum cryptography: Exploiting polynomial multiplication," *ACM Trans. Embedded Comput. Syst.*, vol. 23, pp. 1–23, Mar. 2024.
- [41] Y. Liu, Y. Zhou, S. Sun, T. Wang, R. Zhang, and J. Ming, "On the security of lattice-based fiat-Shamir signatures in the presence of randomness leakage," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1868–1879, 2021.
- [42] Z. Qiao, Y. Liu, Y. Zhou, J. Ming, C. Jin, and H. Li, "Practical public template attacks on CRYSTALS-Dilithium with randomness leakages," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1–14, 2023.
- [43] S. Jendral, J. P. Mattsson, and E. Dubrova, "A single-trace fault injection attack on hedged module lattice digital signature algorithm (ML-DSA)," in *Proc. Workshop Fault Detection Tolerance Cryptography (FDTC)*, 2024, pp. 34–43.
- [44] S. Picek, G. Perin, L. Mariot, L. Wu, and L. Batina, "SoK: Deep learning-based physical side-channel analysis," *ACM Comput. Surv.*, vol. 55, pp. 1–35, Feb. 2023.
- [45] J. Han et al., "Single-trace attack on NIST round 3 candidate Dilithium using machine learning-based profiling," *IEEE Access*, vol. 9, pp. 166283–166292, 2021.
- [46] C. Y. Li et al., "SalsaPicante: A machine learning attack on LWE with binary secrets," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, New York, NY, USA: ACM, 2023, pp. 2606–2620.
- [47] Z. Ni, A. Khalid, D.-e.-S. Kundi, M. O'Neill, and W. Liu, "HPKA: A high-performance CRYSTALS-Kyber accelerator exploring efficient pipelining," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3340–3353, Dec. 2023.
- [48] "7 series FPGAS configurable logic block user guide (UG474)," Xilinx, ug474(v1.8), 2016.
- [49] K. Derya, A. C. Mert, E. Özüürk, and E. Savaş, "CoHA-NTT: A configurable hardware accelerator for NTT-based polynomial multiplication," *Microprocess. Microsyst.*, vol. 89, Mar. 2022, Art. no. 104451.
- [50] R. del Pino, S. Katsumata, T. Prest, and M. Rossi, "Raccoon: A masking-friendly signature proven in the probing model," Cryptology ePrint Archive, Paper 2024/1291, 2024.
- [51] R. del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, and M.-J. Saarinen, "Threshold Raccoon: Practical threshold signatures from standard lattice assumptions," Cryptology ePrint Archive, Paper 2024/184, 2024.



Yijun Cui (Member, IEEE) received the B.Sc. degree in information engineering and the Ph.D. degree in information and communication systems from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2010 and 2020, respectively. From 2014 to 2015, he was a Visiting Ph.D. Student with the Data Security System Group, Centre of Secure Information Technologies, Queen's University Belfast, U.K. Currently, he is an Associate Professor and the Vice Dean with the College of Integrated Circuits, NUAA. His research interests are mainly in hardware security.



Junjie Zhong received the B.E. degree in information engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2022, where he is currently working toward the M.Eng. degree. His research direction is high-performance hardware postquantum cryptographic implementations.



Bei Wang received the Ph.D. degree in information and communication engineering from the University of Science and Technology of China, Hefei, China, in 2021. Currently, she is a Lecturer with the College of Integrated Circuit, NUAA. Her research interests include cryptography and quantum computing. Her primary research focus is on software and hardware acceleration of postquantum cryptographic algorithms and analysis of quantum computing's impact on cryptographic security.



Tianyu Xu received the M.Eng. degree in information engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2024. His research interests include hardware security and cryptographic engineering.



Chenghua Wang (Member, IEEE) received the B.Sc. and M.Sc. degrees from the Southeast University, Nanjing, China, in 1984 and 1987, respectively. He is a Full Professor with the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing. His research interests include testing of integrated circuits and communication systems.



Weiqiang Liu (Senior Member, IEEE) received the B.Sc. degree in information engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, and the Ph.D. degree in electronic engineering from Queen's University Belfast, Belfast, U.K., in 2006 and 2012, respectively. Currently, he is a Professor and the Vice Dean with the College of Electronic and Information Engineering and College of Integrated Circuits, NUAA. He was awarded the prestigious Excellent Young Scholar Award by NSFC in 2020. He serves as an

Associate Editor for IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS, and IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE; a Steering Committee Member of IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS; and a Guest Editor of *Proceedings of the IEEE*. He is the Program Co-Chair of IEEE ARITH 2020 and NANOARCH 2022, and also a TPC Member for DAC, DATE, and ARITH. His research interest focuses on energy efficient and secure computing circuits and systems.